

# API Reference: USB Connectivity

`xm_api_usb.h`에 정의된 **USB 통신 및 데이터 관리 API**에 대한 상세 레퍼런스입니다. XM10은 USB 포트를 통해 두 가지 강력한 기능을 동시에 제공합니다:

1. **MSC (Mass Storage Class)**: USB 메모리(Flash Drive)에 데이터를 파일(bin)로 저장.
2. **CDC (Communication Device Class)**: PC와 가상 시리얼 포트로 연결하여 실시간 데이터 전송 및 디버깅.

이 모듈은 사용자가 원하는 데이터 구조체를 등록하면, 시스템이 백그라운드에서 자동으로 저장하고 전송하는 등록 기반(Registration-based) 자동화 시스템을 갖추고 있습니다.

## 1. 동작 원리 (Operating Principle)

USB 모듈은 사용자의 개입을 최소화하기 위해 **설정(Setup) 후 자동 실행(Automation)** 방식을 따릅니다.

### The Automation Cycle

1. **등록 (Registration)**: 사용자가 `User_Setup()`에서 저장하고 싶은 데이터 구조체(예: `MyData`)의 주소를 시스템에 알려줍니다.
2. **제어 (Control)**: 사용자가 `xm_StartUsbDataLog()`나 시리얼 통신으로 `AGRB MON START`을 호출하여 기능을 켭니다.
3. **자동 처리 (Processing)**: `core_process` 엔진이 2ms마다 `xm_USB_ProcessPeriodic()`을 호출합니다.
  - 이때 시스템은 등록된 구조체의 데이터를 **자동으로 복사**하여 USB 메모리에 쓰거나 PC로 전송합니다.
  - 사용자는 루프마다 `Log()`나 `Send()` 함수를 호출할 필요가 없습니다.

## 2. 데이터 구조 (Enumerations)

### 2.1. Status Types

#### `XmUsbStatus_t` (권장)

함수 실행 결과를 명확히 알리기 위한 반환 타입입니다.

```
typedef enum {
    XM_LOG_STATUS_IDLE,          // 중지됨 (초기 상태)
    XM_LOG_STATUS_LOGGING,      // 정상 로깅 중
    XM_LOG_STATUS_WARNING_QUEUE_FULL, // 큐가 90% 참 (f_write 멈춤 발생 중)
    XM_LOG_STATUS_ERROR_STOPPED, // 큐 오버플로우로 로깅이 강제 중지됨
} XmLogStatus_e;
```

## 3. 함수 상세 (Function Reference)

### 3.1. Data Source Registration (데이터 등록)

가장 먼저 호출해야 하는 함수들입니다. 등록하지 않으면 기본값(XM 전체 구조체)이 사용되거나 아무것도 저장되지 않을 수 있습니다.

#### XM\_SetUsbLogSource

**[MSC용]** USB 파일에 저장할 데이터의 소스(Source)를 지정합니다.

- **Syntax**

```
void XM_SetUsbLogSource(void* data_ptr, uint32_t size);
```

- **Parameters**

- **data\_ptr**: 저장할 사용자 구조체의 주소 (예: &myData)
- **size**: 구조체의 크기 (sizeof(myData))

- **Example**

```
typedef struct { uint32_t time; float angle; } MyLog_t;
MyLog_t myLog;

void User_Setup() {
    // 이 구조체를 매 주기마다 파일에 저장하겠다고 등록
    XM_SetUsbLogSource(&myLog, sizeof(MyLog_t));
}
```

#### XM\_SetUsbStreamSource

**[CDC용]** PC로 실시간 전송할 데이터의 소스를 지정합니다. MSC용 소스와 달라도 상관없습니다.

- **Syntax**

```
void XM_SetUsbStreamSource(void* data_ptr, uint32_t size);
```

- **Parameters**

- **data\_ptr**: 전송할 구조체의 주소
- **size**: 구조체의 크기

- **Note**: 이 함수로 등록된 데이터는 **StartUsbStream()** 호출 시 **바이너리(Binary)** 형태로 PC에 전송됩니다. (Serial Plotter 등에 적합)

---

### 3.2. MSC Control (데이터 로깅)

USB 메모리에 파일을 생성하고 데이터를 기록하는 기능입니다.

### XM\_StartUsbDataLog

데이터 로깅을 시작합니다.

- **Syntax**

```
bool XM_StartUsbDataLog(const char* sessionName, const char* metadata);
```

- **Parameters**

- **sessionName**: 생성할 폴더의 접두어. (예: "Walk" -> Walk/data\_000\_part\_000.bin, data\_001\_part\_000.bin...)
- **metadata**: 파일 헤더(첫 줄)에 기록할 문자열. bin 컬럼명 등을 적기에 좋습니다. (NULL 가능)

- **Returns**: **true** (시작 성공), **false** (USB 없음 또는 오류)

- **Example**

```
// 버튼을 누르면 "Test_xx.csv" 파일을 만들고, 헤더를 "Time,Angle"로 적음
if (GetButtonEvent(XM_BTN_1) == XM_BTN_CLICK) {
    XM_StartUsbDataLog("Test", "Time_ms, Angle_deg");
}
```

### XM\_StopUsbDataLog

로깅을 중단하고 파일을 저장(Close & Sync)합니다. **[주의]** USB를 뽑기 전에 반드시 이 함수를 호출해야 데이터가 깨지지 않습니다.

- **Syntax**

```
void XM_StopUsbDataLog(void);
```

### XM\_IsUsbLogReady

USB 메모리가 인식되었고 파일 시스템이 준비되었는지 확인합니다.

- **Syntax**

```
bool XM_IsUsbLogReady(void);
```

- **Returns**: **true** (준비됨), **false** (연결 안 됨)

## 3.3. CDC Control (디버그 및 스트리밍)

PC와 시리얼 통신을 수행합니다.

### XM\_SendUsbData

PC 터미널(TeraTerm 등)로 \*\*데이터 구조체\*\*를 전송합니다. 'AGRB MON START'를 사용하지 않고 사용자가 원하는 대로 데이터를 전송하고자 할 때 제공하는 함수입니다.

- **Syntax**

```
bool XM_SendUsbData(const void* data, uint32_t len);
```

- **Parameters**

- `message`: 전송할 문자열 (Null-terminated)

- **Example**

```
typedef struct { uint32_t time; float angle; } MyLog_t;  
MyLog_t myLog;  
XM_SendUsbData(&myLog, sizeof(MyLog_t));
```

### XM\_SendUsbDebugMessage

PC 터미널(TeraTerm 등)로 \*\*문자열(Text)\*\*을 전송합니다. `printf`와 유사하게 디버깅 용도로 사용합니다.

- **Syntax**

```
bool XM_SendUsbDebugMessage(const char* message);
```

- **Parameters**

- `message`: 전송할 문자열 (Null-terminated)

- **Example**

```
char buf[64];  
sprintf(buf, "Current State: %d\r\n", current_state);  
XM_SendUsbDebugMessage(buf);
```

### XM\_IsUsbStreamConnected

USB 케이블이 PC에 연결되어 가상 시리얼 포트가 열렸는지 확인합니다.

- **Syntax**

```
bool XM_IsUsbStreamConnected(void);
```

### XM\_GetUsbData

PC로부터 데이터를 수신합니다. (키보드 입력 등)

- **Syntax**

```
uint32_t XM_GetUsbData(void* buffer, uint32_t max_len);
```

- **Returns:** 실제로 읽어온 바이트 수

---

## 3.4. System Interface

### XM\_USB\_ProcessPeriodic

[시스템 내부용] USB 로깅 및 스트리밍 로직을 처리하는 함수입니다. `core_process`에 의해 자동으로 호출되므로 **End User**는 직접 호출할 필요가 없습니다.

- **Syntax**

```
void XM_USB_ProcessPeriodic(void);
```